Guidelines for Writing Fixes - Base

Guidelines for writing fixes - Standard
Guidelines for writing fixes - Simplified
Guidelines for importing simplified fixes table into MadCap Flare

Guidelines for writing fixes - Standard

1. Title of the Fix

- Always start with "*Fix for*".
- Try to avoid mentioning "crash".
- Keep it concise and descriptive.
- Include the specific area or feature affected by the fix.
- Example:
  - "*Fix for episode title formulas for episodes with many transmissions*"
  - *"Fix for product first future transmission dates not ordered chronologically"*

Other examples here: Fixed issues - Base 2024r6

2. Issue Description

- Clearly describe the problem that existed before the fix.
- Include specific scenarios or actions that triggered the issue.
- Mention any error messages or system crashes, if applicable.
- If necessary, give some context for the area of the fix. For example, to give a link to the KB or RN. Fix for exploitation rights filter as REST search parameter
- Example: "*Previously, a crash would occur when defining the episode title formula with a placeholder on the episode of a series and executing the calculation of the placeholders*."

Fix for options group not overwritten in template layout management

Other examples here: Fixed issues - Base 2024r6

3. Steps to Reproduce:

- Provide a step-by-step guide on reproducing the issue for complicated scenarios, such as crashes.

>>> This is not applicable for simple fixes such as this: MGXPRODUCT-15452. A couple of sentences with a screenshot are then enough, like this: Fix for background colour applied incorrectly in layouts

- Use bullet points or numbered lists for clarity.
- Example:

Previously, a crash would occur when

1. *defining the episode title formula with a placeholder on the episode of a series;*
2. *making sure that the episode was planned a lot, for example, having more than 1000 transmissions;*
3. *executing the calculation of the placeholders, for example by including the* ***Different from episode title*** *tag (****differentTitleThanFormula****) in the XML filter of a report definition and running the report.*

[...] Fix for episode title formulas for episodes with many transmissions

Other examples here: Fixed issues - Base 2024r6

4. **Cause of the Issue:**

- Explain why the issue was occurring.
- Provide technical details if necessary.
- Example: "*This happened because the tag checked if the title of the episode was different from the one calculated by the formula. For that last check, it calculated the title again, which used to retrieve all transmissions of the episodes and pick the first. If there were a lot, the query would go out of memory*."

5. **Solution**:

- Describe what was done to fix the issue – only if applicable.
- Be specific about the changes made to achieve that – if applicable.
- Example:
  - "*This issue has been fixed by adjusting the way titles are calculated to prevent memory overload.*"
  - *"From this version, the **Default title** on product level has been removed from the layout configuration of the trailer campaign navigator and only the specific one remains."*

6. **References**:

- Reference the ticket at the end between brackets.
- Example: "(MGXPRODUCT-15754)"

Check the WHATS'ON BASE RELEASE NOTES container of the release notes space for more examples of Base release notes.

Guidelines for writing fixes - Simplified

Overview

To streamline and speed up our process while managing workload efficiently, bug fix release notes will be more concise and structured. Instead of long descriptions, each fix will be summarized in one to two sentences via AI (ChatGPT) and documented in a structured Confluence table with filtering capabilities. This solution is being adopted as of release *2025r1* to ensure faster delivery of release notes.

Release notes format

Bug fix release notes will be recorded in a Confluence table under the page **Fixed issues**. The table will have the following columns:

- **Issue key** – The JIRA ticket number (MGXPRODUCT ID).
- **Description** – A brief (one- to two-sentence) explanation of the fix.
- **Module** – The affected module.
- **Labels** – Keywords to enable filtering and categorization.

The table will be placed inside a **Table Filter Macro** in Confluence to allow easy filtering and sorting.

Step-by-step process for writing bug fixes

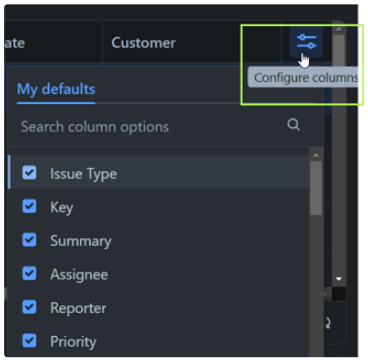
1. **Gather JIRA issues**

Search for all bug issues for the release in JIRA using the following query in JQL:

```
project = "MGXPRODUCT"
AND type = Bug
AND status IN (Done, Delivered)
AND fixversion = "VERSION_PLACEHOLDER"
AND (labels NOT IN (RN_NN) OR labels IS EMPTY)
```

Generate an Excel export with the following columns:

- **Key** (JIRA ticket number)
- **Summary**
- **Description** (used to generate the release note)
- **Release notes description**



To only export the chosen columns, select **Export** > **Export Excel CSV (my defaults)**.

Write fix release notes

Final Optimized GPT Prompt for Future Use:

Here is an Excel file with a list of JIRA issues. Please process it and generate a Confluence-style table with the following columns: ***Issue key, Description, Module, and Keywords****. Follow this exact process:*

Step 1: Convert the JIRA Export for Parsing

- Extract **Issue Key, Summary, Description and Release notes description** from the Excel file.
- Convert it into a structured format (TXT or JSON) that is **easier to parse** for AI processing.
- Ensure descriptions are cleaned up (removing artifacts, formatting inconsistencies, and unnecessary JIRA metadata).
- The parsed file should be structured so that it can be easily read and used for summarization.

Step 2: Summarize Descriptions into Release Notes

- Use the **parsed file** from Step 1 as the source for summarization.
- Fetch the info from the description columns of the parsed excel table following this logic:
  - If there is content in the **Release notes description** column, take that content into account and ignore the content from the **Description** field.
  - If there is no content in the **Release notes description** column (therefore, if empty), take the content from the **Description** field into account.
- Rewrite the relevant description into a concise, **customer-friendly release note,** in the following format
  - **Start with**: 'Previously,' explaining the issue in simple terms.

- **End with**: 'This issue has been fixed in this version.'
- Retain key details to provide meaningful context. Do not oversimplify.
- If there are step-by-step reproduction scenarios, ignore them and reformulate the content logically without using the step-by-step format.

Sometimes you might encounter the following issues:

- Not all tickets had clean, structured inputs.
- Some release note fields started with “This issue has been fixed…” with limited background.
- Without explicit As-is/To-be or walkthroughs, I defaulted to conservative summarization.

In these cases, do the following:

Always scan for functional depth, not just release note phrasing.

- Deprioritize brevity in favor of precision and traceability.
- Match the tone and rigor of the *Base2025r4_TrainingFixes.json* style exactly.

Step 3: Assign Modules and Keywords (WHATS’ON Guidelines)

Modules

- Assign **one or more modules** based on the affected system areas.
- Use the **official module labels** from WHATS’ON.

Description	Label
Alternative Scheduling	`alternative_scheduling_module`
Aspose	`aspose_module`
Transmission Automation Integration	`as_run_log_module`
Secondary Events As-Run Log	`as_run_log_secondary_events_module`
Strategic Planning & Budget Forecasting	`budget_simulation_module`
Bumblebee Reports	`bumblebee_module`
Sponsoring & Bumper Autoscheduling	`bumper_autoslotter_module`
Business API	`business_api_module`
Business Layer	`business_layer_module`
Acquisition, Screening & Buying Order Management	`buying_order_module`
BXF Automation Integration	`bxf_module`
Catch-up Scheduling	`catch_up_module`

Cloud	`cloud_module`
Commercial Integration	`commercial_integration_module`
Commercial Spot Management	`commercial_module`
Compliance	`compliance_module`
Configurable REST	`configurable_rest_module`
Continuity Grid Planning	`continuity_grid_planning_module`
Continuity Scheduling	`continuity_module`
Contract & Rights Management	`contract_module`
Use Multi-runs in Contract	`contract_multiruns_module`
Underlying Rights (Contract)	`contract_underlying_rights_module`
Music & Copyright Reporting	`copyright_module`
WHATS'ON Core	`core_module`
Statistics & Analysis	`cost_analysis_module`
CSA	`csa_module`
Curation Management	`curation_module`
Dashboard	`dashboard_module`
Advanced Graphics and Dynamic Branding	`dynamic_branding_module`
External Workflow Module	`external_workflow_module`
Financial Stock Management	`financial_stock_module`
Model-Based API	`generic_importer_module`
Programme Grid Planning	`grid_planner_module`
Business Datasets	`insight_module`
Interstitial Integration	`interstitial_integration_module`

License	`license_module`
Lifeguard	`lifeguard_module`
Scheduling	`linear_scheduling_module`
Mediator	`mediator_module`
Media Asset Management (MM2)	`mm2_module`
Multiple Local Currencies	`multi_currency_module`
Multi Frame Rate Support	`multi_frame_rate_module`
Music Rotation Integration	`music_scheduling_integration_module`
Music Clip Management	`music_scheduling_module`
Network Synchronization	`network_synchronization_module`
On-demand Scheduling	`on_demand_module`
Web Connectivity (27Q2)	`online_api_module`
SOAP/WEB Service Interface Framework	`open_interface_module`
ORAD	`orad_module`
Parent-child Channels	`parent_child_channels_module`
Power Report Builder	`power_report_module`
Product Extract Management	`product_extract_management_module`
Program Versions	`product_version_module`
Program Guide & EPG	`program_guide_module`
Program Management	`program_module`
PROMOPLAN	`promoplan_module`
Quota Reporting	`quota_reporting_module`
Regional Commercial Planning	`regional_spot_module`
Reporting & Exporting Engine	`report_module`

Rights Out	`rights_out_module`
Return on Investment	`roi_module`
Running Order	`running_order_module`
SGT	`sgt_module`
Sports Team Management	`sport_module`
Sports Rights Management	`sport_rights_module`
Traffic & Material Handling	`traffic_module`
Promotion Optimiser	`trailer_auto_slotter_module`
Promotion & Interstitial Campaign Management	`trailer_module`
Encoding and Transcoding	`tx_encoding_module`
On-demand Publishing	`tx_publishing_module`
Scheduling Artist	`tx_auto_slotter_module`
Continuity Artist	`schedule_finalization_module`
Video Player Integration Framework	`videoplayer_integration_module`
Web Connectivity	`web_server_module`
Workflow Interstitials	`workflow_interstitial_module`
Workflow Media Management	`workflow_mm_module`
Workflow Engine	`workflow_module`
WHATS'ON Web	`wow_module`

Keywords

Each release note must have **at least 2 keywords**, assigned using these rules:

1. **Concept Keyword** → Use the **plural name** of the concept it refers to.
   - Example: If the release note is about **contracts**, assign `contracts`.
   - Example: If the fix is related to **scheduling**, assign `scheduling`.
2. **Application Keyword** → Assign the **correct application name** if applicable.
   - Example: If the fix affects **contract navigator**, assign `contract_navigator`.
   - Example: If it affects the **copyright sheet navigator**, assign `copyright_sheet_navigator`.

- **Avoid incorrect terms** (e.g., **do NOT use** `copyright_navigator`).

3. **Drop-down Lists & Gadgets** → If the fix involves UI elements like drop-downs or gadgets, assign:
   - `drop-down_lists` (for drop-down-related fixes).
   - `gadgets` (if related to gadgets).
4. **Upgrade Label** → If the fix **affects the upgrade process**, add `upgrade` as a keyword.
5. **Remove Version-Specific Labels**
   - Do **NOT** include version labels like `base2024r3`.

Step 4: Generate the Final Confluence Table

- Format the final output as a **structured table** ready for Confluence.
- Ensure it is clean, accurate, and fully structured for easy import into Confluence.

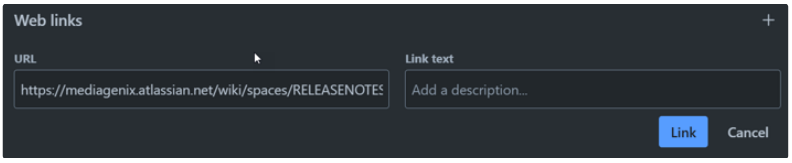
Insert the table into Confluence

1. Copy and paste the generated table into the **Fixed issues** page in Confluence.
   - Add **Module** and **Labels** columns.
   - Wrap the table inside a **Table Filter Macro** by following these steps:
     - Click **Insert elements** (the plus icon in the toolbar).
     - Select **Other macros**.
     - Search for **Table Filter** and select it.
     - In the **Insert "table filter" macro** dialogue box, select **Filtration panel**.
     - Place the Confluence table inside the **Table Filter Macro** to enable filtering and sorting.

Add JIRA links for traceability

To ensure easy reference between Confluence and JIRA:

- **Add anchors (bookmarks) next to each Issue key** in the Confluence table:
  - Place the cursor before the **Issue key** in the table.
  - Click **Insert more content** > **Other macros** > **Anchor**.
  - Assign a unique anchor name (e.g., the ticket number "*MGXPRODUCT-12345*").
- **In JIRA, add a web link** to the anchor of the Confluence page where the release note is documented:
  - Open the JIRA issue.
  - Click **More** > **Link** > **Web Link**.
  - Paste the Confluence URL with the anchor at the end (e.g., https://mediagenix.atlassian.net/wiki/spaces/RELEASENOTES/pages/1438089319/25r1+-+Fix+container**#MGXPRODUCT-20417**).
  - In the **Link text** box, adjust the title if necessary, make sure the titles match the Excel sheet.



Example

See Fixed issues - Base 2025r1 to check how the bugs for 2025r1 look like.

Review descriptions

- Compare the information from the Jira ticket and the description created by ChatGPT.

- Make sure bugs are suppose to stay as bugs and are not features. When something new other than fixing the issue has been added, for example, a search field that has become a read-only field, or something that has been added in order to fix the bug, then this is a feature. In this case, create a confluence page (copy to Documentation as well and place it in the backlog) and write the bug ticket as a feature in Confluence.
- Adjust the formatting in the description.
- Add modules and labels to the **Module** and **Label** columns.
- Add all modules at the bottom of the Confluence page to help users filter the table outside of the Confluence page.
- Add links from the KB and previous release notes, when writing a bug that might require to mention several modules, tabs or navigators, in more than two lines.

Before Preview

After finishing with the review of the bugs in the table in Confluence and before moving the table to Preview, go back to ChatGPT and ask the tool to compare the first versions that it created in step **Write fix release note** and this last version reviewed by the Doc team.

Ask the tool to explain what it did wrong and then ask to memorize this so that we can build from there.

Move Confluence page to Preview

Move Confluence page to Preview once all steps have been completed.

Make sure all labels and modules are included, and all descriptions are correct.

Guidelines for importing simplified fixes table into MadCap Flare

Step 1: Export from Confluence to Word

1. Navigate to the Confluence page containing the Fixed Issues for the release.
2. Click on the **More options (•••)** menu at the top-right corner of the page.
3. Select **Export** > **Word** to generate a Word document (.docx) of the page.
4. Save the exported Word document to your local machine in a designated folder (e.g., `Downloads` or a relevant documentation folder).

Step 2: Import into MadCap Flare

1. Open **MadCap Flare** and navigate to the **Content Explorer**.
2. Locate the correct **Release Notes** folder under `Content/ReleaseNotes/` for the corresponding software version.
3. Click **Project** > **Import** > **MS Word Documents...**
4. In the **Word Import Editor**, ensure the following settings are applied:

General

- **Import to folder:** Select the correct release folder (e.g., `Content\ReleaseNotes\RN2024`).
- Click **Add File** and select the exported Word document.
- Verify that the document appears in the import list.

Styles

- **Associate a stylesheet:** Use `C:\TFS\MFWONKB2024r13_icons\Content\wOn.css`.
- Click **Discard MS Word styles** to ensure that the Flare stylesheet is applied consistently.

Advanced Options

- **Convert inline formatting to CSS styles** (to maintain consistency).
- **Automatically set topic title** (recommended).
- **Avoid empty topics threshold:** Set to `50` characters.
- **Tables:**
  - Set **first row of each table** as a **header row**.
  - Apply a **selected table stylesheet** to all imported tables (`w0n.css`).
- Click **Finish**.

Step 3: Import and review the topic

1. Click **Import** to import the document into Flare.
2. Open the newly created topic. .
3. Check for:
   - Correct formatting (headings, paragraphs, lists, tables).
   - Proper application of the stylesheet (`w0n.css`).
   - Correct topic title and filename length.
4. Apply the correct table style. If necessary, recreate the header.
5. Remove **Module** and **Keywords** columns